Project for Sixth Semester of Bachelor of Information Technology

Kishan Sahayatri



Aashutosh Lama (S.N.)

Anish Thapa (S.N.)

Prasan Poudel (S.N.)

KIST College of Information Technology

Faculty of Science and Technology

Purbanchal University, Nepal

2026

DECLARATION

Our project titled **"Kishan Sahayatri"** is a consequence of our own work and indebtedness to other work publication and references have been wholly acknowledged. If we have been found guilty of copying any other report or information and showing as our original work, we shall be punishable by University, which may include failing in examination or any other punishment that Purbanchal University decide, which we understand very nicely so, it is a pure effort of our own.

We further certify that this project is submitted in fulfillment of requirement for the project marks of Bachelor of Information Technology (BIT) 6th Semester. So, it is our original work and has not been submitted for award of any other degree or title.

S. N	NAME	Reg. No.	Symbol No.	Signature
1.	Aashutosh Lama	058-3-2-02413-2023		
2.	Anish Thapa	058-3-2-02412-2023		
3.	Prasan Poudel	058-3-2-02440-2023		

RECOMMENDATION

CERTIFICATE

This project entitled **"Kishan Sahayatri"** prepared and submitted by **Aashutosh Lama, Anish Thapa and Prasan Poudel** has been examined by us and is accepted by the degree of Bachelor of Information Technology by Purbanchal University.

Mukunda Raj Joshi	Signature	Date: Oct 7, 2026
External Examiner		
Bishal Khatri	Signature	Date: Oct 7, 2026
Project Teacher		
Deepak Khadka	Signature	Date: Oct 7, 2026
Coordinator, BIT		

ACKNOWLEDGEMENTS

We take this opportunity to express our profound appreciation and regards to the Information Technology (IT) department for this commendable guidance, monitoring and constant encouragement throughout the course of this project. The help and guidance given by shall carry us a long way, in the journey on which we are about to embark.

We also take this opportunity to express a deep sense of gratefulness to our subject teacher as well as our project teacher **Mr. Bishal Khatri** for his amiable support, valuable information and guidance, which helped us in completing this task throughout its various stages by giving us his precious time. We are also indebted to all members of KIST College, for the valuable suggestions and help provided by them in their respective fields. We are grateful for their cooperation during the period of our project.

Finally, we would also like to express lots of thanks to **PURBANCHAL UNIVERSITY** for designing such a wonderful course structure. It will help us to get more knowledge in the field of Information Technology & help us to have a bright future in the field of technology. We hope our university will accept this attempt as a successful project.

ABSTRACT

This documentation presents the design and development of **Kishan Sahayatri**, a machine learning-based web application implemented using Python (Flask), React.js, and scikit-learn, aimed at supporting agricultural decision-making through intelligent prediction models. The system provides an interactive, user-friendly interface that allows farmers and agricultural stakeholders to obtain data-driven recommendations for crop selection, fertilizer management, and yield estimation by providing relevant soil, environmental, and cultivation parameters.

The application integrates three independent machine learning models into a unified web platform. The Crop Prediction model utilizes a Random Forest classifier trained on soil nutrient levels (N, P, K), temperature, humidity, soil pH, and rainfall data to recommend the most suitable crop for cultivation. The Fertilizer Prediction model employs a Decision Tree classifier that analyzes soil characteristics, nutrient deficiencies, and crop type to suggest the appropriate fertilizer for optimal plant growth. The Yield Prediction model applies a regression-based approach to estimate expected crop production using historical agricultural data, cultivated area, rainfall, temperature, pesticide usage, crop type, and year as input parameters.

All three models are serialized using joblib and served through a RESTful Flask API backend, which processes user inputs, invokes the respective trained model, and returns prediction results to the React.js frontend in real time. The system is designed to reduce dependency on traditional trial-and-error farming practices, minimize fertilizer misuse, and provide smallholder farmers with accessible, cost-effective decision support without requiring any domain expertise.

Keywords: *Machine Learning, Crop Prediction, Fertilizer Prediction, Yield Prediction, Random Forest, Decision Tree, Flask REST API, React.js, Agricultural Decision Support System, scikit-learn*

TABLE OF CONTENTS

DECLARATION II

RECOMMENDATION III

CERTIFICATE IV

ACKNOWLEDGEMENTS V

ABSTRACT VI

LIST OF FIGURES X

LIST OF TABLES XI

CHAPTER 1: BACKGROUND 1

CHAPTER 2: SIGNIFICANCE 8

2.1 Challenges Present in the Absence of this Software 8

2.2 Proposed Solutions to Address the Identified Challenges 8

CHAPTER 3: LITERATURE REVIEW 10

3.1 Historical Context 10

3.2 Software-Based Agricultural Decision Support Systems 10

3.3 Notable Research Contributions 11

CHAPTER 4: METHODOLOGY 13

4.1 Requirement Analysis 13

4.1.1 Functional Requirements 13

4.1.2 Non-Functional Requirements 14

4.2 System Design 15

4.2.1 System Architecture 15

4.2.2 Machine Learning Pipeline 16

4.2.3 Use-Case Diagram 18
4.2.4 Sequence Diagram 19
4.2.5 Data Flow Diagram 20
4.2.6 Algorithms 21
4.2.7 System Flowchart 23
4.3 Tools and Technologies 24
4.4 Development Model 25
4.5 Model Training and Evaluation Strategy 25
4.5.1 Crop Prediction Dataset 25
4.5.2 Crop Data Preprocessing 26
4.5.3 Crop Prediction Findings 27
4.5.4 Crop Model Evaluation 28
4.5.5 Yield Prediction Dataset 30
4.5.6 Yield Data Preprocessing 30
4.5.7 Yield Prediction Findings 31
4.5.8 Yield Model Evaluation 33
4.6 Testing Strategy 33
4.6.1 Unit Testing 33
4.6.2 Integration Testing 34
4.6.3 System Testing 34
4.6.4 Test Cases 34
4.7 Deployment and Maintenance 35
4.8 Project Management 35
4.8.1 Gantt Chart 36

CHAPTER 5: SNAPSHOTS 37

CHAPTER 6: CONCLUSION 40

References 42

LIST OF FIGURES

FIGURE 1: SYSTEM ARCHITECTURE 15
FIGURE 2: CROP PREDICTION PIPELINE 16
FIGURE 3: YIELD PREDICTION PIPELINE 17
FIGURE 4: USE-CASE DIAGRAM 18
FIGURE 5: SEQUENCE DIAGRAM 19
FIGURE 6: CONTEXT-LEVEL DFD 20
FIGURE 7: LEVEL-1 DFD 20
FIGURE 8: SYSTEM FLOWCHART 23
FIGURE 9:LOGIN PAGE 37
FIGURE 10: MAIN PAGE 37
FIGURE 11: CROP PREDICTION 38
FIGURE 12: YIELD PREDICTION 38
FIGURE 13: CONFUSION MATRIX(CROP PREDICTION) 39

LIST OF TABLES

TABLE 1:TOOLS AND TECHNOLOGY 7
TABLE 2: CHALLENGES AND PROPOSED SOLUTION 9
TABLE 3: FUNCTIONAL REQUIREMENTS 14
TABLE 4: NON-FUNCTIONAL REQUIREMENTS 14
TABLE 5: TOOLS AND TECHNOLOGIES USED 25
TABLE 6: CROP PREDICTION DATASET DESCRIPTION 26
TABLE 7:CROP PREDICTION FINDINGS 28
TABLE 8: CROP PREDICTION MODEL RESULTS 28
TABLE 9:YIELD PREDICTION DATASET DESCRIPTION 30
TABLE 10:YIELD PREDICTION FINDINGS 32
TABLE 11:YIELD PREDICTION MODEL RESULTS 33
TABLE 12: TESTING CASES 35
TABLE 13: PROJECT DEVELOPMENT SCHEDULE (GANTT CHART) 36

CHAPTER 1: BACKGROUND

1.1 Introduction

Agriculture is one of the most important sectors for food production and economic development in many countries, including Nepal. Agricultural outcomes depend on many interacting factors, such as soil nutrients, temperature, humidity, soil pH, rainfall, crop type, geographical area, and the use of agricultural inputs like pesticides. Two decisions are especially important for farmers and agricultural planners: which crop is suitable for the given conditions, and how much yield can be expected from a crop. These decisions have traditionally relied on personal experience, local knowledge, seasonal patterns, and expert consultation. Such methods are valuable, but they are difficult to apply consistently when many variables must be considered together, and they rarely give quantitative estimates.

Machine Learning (ML) provides a way to learn relationships between input variables and agricultural outcomes from historical data. Classification techniques can predict a suitable crop from soil and climate conditions, while regression techniques can estimate crop yield as a numerical value. However, models built in development environments such as Jupyter Notebook cannot be used directly by ordinary users. They must be made accessible through an application with a simple interface.

Kishan Sahayatri is a machine-learning-based agricultural web application developed to address this gap. It provides two prediction modules through a web interface:

a. **Crop Prediction** which recommends a crop from nitrogen, phosphorus, potassium, temperature, humidity, soil pH, and rainfall.
b. **Crop Yield Prediction** which estimates yield in kg/ha from area, crop/item, average annual rainfall, pesticide usage, and average temperature.

The system is organized into four layers. The presentation layer is built with React. The application layer uses Node.js and Express.js to handle authentication and API routes. MongoDB serves as the database layer for users and application data. The machine-learning layer uses Python, FastAPI, Scikit-learn, and Joblib to load the trained models and serve predictions over HTTP. Separating the ML service from the web application allows each part to be developed, tested, and maintained independently.

1.2 Problem Statement

Agricultural decision-making involves a large number of variables, including soil nutrient levels, climate conditions, crop type, and location. Analyzing these variables manually is difficult, particularly when a large amount of historical data is involved. As a result, crop selection and yield estimation often depend heavily on experience and may not be supported by data.

Although machine-learning techniques can process such data, several problems prevent farmers and agricultural users from benefiting from them:

a. Trained models developed in notebooks are not directly accessible to non-technical users.
b. Prediction functionality is often spread across separate scripts and tools rather than a single platform.
c. Users need a simple way to enter parameters, obtain a prediction, and understand the result.
d. Models need to be served through an API so that a web application can send inputs and receive predictions reliably.

Therefore, there is a need for an integrated web application that accepts agricultural inputs through a simple interface, validates them, processes them with trained machine-learning models, and presents crop and yield predictions in an understandable format, while managing user accounts and application data securely.

1.3 Objective of the Project

General Objective

To develop a web-based machine-learning application that provides crop and crop-yield predictions from agricultural and environmental data.

Specific Objectives

a. To develop a user-friendly web interface using React.
b. To develop a backend with user authentication using Node.js, Express.js, and JWT.
c. To use MongoDB for storing user information and application data.
d. To preprocess the crop and yield datasets before model training.
e. To develop a crop classification model using soil and climate parameters.
f. To develop a crop yield regression model.
g. To compare multiple machine-learning algorithms using suitable performance metrics.
h. To serve the trained models through FastAPI endpoints.
i. To integrate the frontend, backend, database, and ML service into one working system.
j. To display prediction results clearly to the user.

1.4 Scope of the Project

The scope of Kishan Sahayatri includes:

a. User registration, login, and access to a dashboard.
b. Crop prediction using 22 crop classes, based on a dataset of 2,200 records.
c. Crop yield estimation using a dataset of 25,932 records after duplicate removal.
d. Data preprocessing, model training, comparison, and evaluation.

e. Saving trained models together with their preprocessing objects (scaler, label encoder, feature columns) for reuse.
f. Serving predictions through FastAPI.
g. Backend services using Node.js and Express.js, with data storage in MongoDB.
h. Displaying prediction results through the React frontend.

The architecture is modular, so it can be extended in the future with additional modules such as fertilizer recommendation, disease detection, weather integration, soil-data integration, and agricultural advisory services.

1.5 Limitations

a. Predictions depend on the quality and characteristics of the training datasets.
b. The crop dataset covers only 22 crop classes and does not represent every possible crop.
c. The yield dataset is based on historical records from particular geographical areas, so performance on new locations may differ from the reported test results.
d. Real-world agricultural outcomes depend on factors that are not included in the current models, such as soil type, seed variety, pests, and farming practices.
e. Predictions should be treated as decision-support information, not as guaranteed outcomes.
f. The current implementation does not include a fertilizer recommendation model.
g. The system relies on the ML service being available; if FastAPI or the model files are unavailable, predictions cannot be produced.

1.6 Features

a. **User Registration and Login**
   Users can create an account and log in. Authentication is handled by the Node.js/Express backend using JWT.

b. **Dashboard**

A dashboard gives access to the prediction modules and displays results.

c. **Crop Prediction**

The user enters nitrogen, phosphorus, potassium, temperature, humidity, soil pH, and rainfall. The input is scaled with the saved scaler and passed to the Logistic Regression model, and the predicted class is decoded with the label encoder to give the crop name.

d. **Crop Yield Prediction**

The user enters area, crop/item, average annual rainfall, pesticide usage, and average temperature. The Decision Tree pipeline, which handles categorical encoding internally, returns the estimated yield in kg/ha.

e. **Input Validation**

Inputs are validated in the interface and before prediction, and an error message is shown when the data is invalid.

f. **API-Based ML Integration**

FastAPI exposes the trained models as HTTP endpoints, so the web application can request predictions in JSON format.

g. **Data Management**

MongoDB stores user information, prediction history, and other application-related data.

h. **Responsive Web Interface**

The React frontend provides an interactive interface for entering parameters and viewing results.

1.7 Workflow of the Project

The overall workflow of the system is:

User → React Frontend → Node.js/Express Backend → FastAPI ML API → Trained ML Model → Prediction → Backend → React Frontend → User

Crop Prediction Workflow

a. The user enters agricultural data in the React frontend.
b. React sends an HTTP request to the Node.js/Express backend.
c. The backend verifies the user's JWT token and validates the input.
d. The backend forwards the input to FastAPI.
e. FastAPI applies the saved scaler and runs the Logistic Regression model.
f. The predicted class is decoded with the label encoder to obtain the crop name.
g. The result is returned to the backend, saved to MongoDB, and sent to React.
h. React displays the predicted crop to the user.

Yield Prediction Workflow

a. The user enters area, crop/item, rainfall, pesticide usage, and temperature.
b. The frontend sends the request to the backend, which verifies the token and validates the input.
c. The backend forwards the input to FastAPI.
d. The Decision Tree pipeline performs encoding internally and predicts the yield.
e. The estimated yield in kg/ha is returned through the backend and displayed by React.

In summary, React handles the user interface, Node.js/Express handles application-level operations, MongoDB stores application data, and FastAPI handles machine-learning inference.

1.8 Introduction to the Tools and Language Used

Tool / Technology	Purpose in the Project
React	Frontend user interface, forms, and result display
Node.js	Runtime environment for the backend
Express.js	REST API routes and authentication middleware
MongoDB	NoSQL database for users and prediction history
JWT	Token-based user authentication
FastAPI	Serves trained ML models as API endpoints
Python	Data preprocessing, model development, and ML serving
Pandas	Dataset loading, cleaning, and analysis
NumPy	Numerical operations during data processing
Scikit-learn	Preprocessing, model training, and evaluation
Joblib	Saving and loading trained models and preprocessors
Jupyter Notebook	Data analysis, model training, and experimentation
Git/GitHub	Version control for source code

Table 1:Tools and Technology

CHAPTER 2: SIGNIFICANCE

2.1 Challenges Present in the Absence of this Software

Several challenges exist without an integrated agricultural prediction system:

a. **Manual Decision Making:** Agricultural decisions rely heavily on personal experience and traditional knowledge, which may not systematically account for multiple quantitative variables simultaneously.

b. **Difficulty in Analyzing Multiple Variables:** Agricultural datasets contain multiple environmental and agricultural variables. Manually analyzing the relationships between nitrogen, phosphorus, potassium, temperature, humidity, pH, rainfall, and crop outcomes is difficult and error-prone.

c. **Lack of Accessible ML Tools:** A trained machine-learning model in a development environment such as Jupyter Notebook is not directly accessible to ordinary users. Farmers cannot interact with Python scripts or machine-learning code directly.

d. **Difficulty Estimating Yield:** Historical relationships between crop production variables and actual yield figures can be complex and difficult to analyze manually, especially across many geographical areas and crop types.

e. **Lack of Centralized Platform:** Without an integrated application, prediction functionality remains distributed across separate tools, scripts, and notebooks that are not accessible to non-technical users

2.2 Proposed Solutions to Address the Identified Challenges

Kishan Sahayatri addresses these challenges by providing an integrated solution. The table below summarizes the relationship between identified challenges and the proposed solutions:

Challenge	Proposed Solution
Manual analysis of agricultural variables	ML-based automated prediction
Many input variables to process	Automated model preprocessing pipeline
Inaccessible ML models in notebooks	FastAPI model-serving layer

Table 2: Challenges and Proposed Solution

CHAPTER 3: LITERATURE REVIEW

3.1 Historical Context

Agriculture has been the foundation of human civilization for thousands of years, and the pursuit of improving farming outcomes through systematic knowledge has existed just as long. Historically, agricultural decisions were guided by generational wisdom, seasonal observation, and manual soil assessment techniques passed down through farming communities. The introduction of the Green Revolution in the 1960s marked a significant turning point, bringing high-yield crop varieties, chemical fertilizers, and irrigation systems that dramatically increased food production across South and Southeast Asia, including Nepal. However, this era also gave rise to persistent problems such as widespread fertilizer misuse, soil nutrient depletion, and environmental degradation that continue to affect farming communities to this day.

As computational technology advanced through the late twentieth century, researchers began exploring data-driven approaches to agricultural management. Early expert systems attempted to encode agricultural knowledge into rule-based decision trees, but their rigidity and inability to generalize across different environmental conditions limited their practical impact. It was not until the widespread availability of large agricultural datasets and the maturation of machine learning algorithms in the twenty-first century that truly adaptive, data-driven agricultural prediction became feasible forming the technological foundation upon which systems like **Kishan Sahayatri** are built.

3.2 Software-Based Agricultural Decision Support Systems

The emergence of digital agricultural tools has produced a wide range of software solutions aimed at supporting farmers and agricultural professionals. These systems can broadly be categorized into three types: advisory platforms, remote sensing tools, and machine learning-based prediction systems:

a. **Advisory Platforms** such as Plantix, AgroStar, and Digital Green provide farmers with pest identification, crop health monitoring, and general farming advice through mobile applications. While these platforms have achieved significant reach in countries like India, they primarily rely on image recognition and community-based knowledge sharing rather than predictive modeling based on soil and environmental parameters. Their applicability to structured prediction tasks such as crop recommendation or yield estimation is therefore limited.
b. **Remote Sensing and GIS-Based Tools** utilize satellite imagery and geographic information systems to monitor crop health, detect irrigation needs, and assess land use patterns at scale. Organizations such as NASA and the Food and Agriculture Organization (FAO) have developed open datasets and tools based on this approach. However, these systems are primarily designed for policy-level analysis and large-scale agricultural planning rather than individual farmer decision support.
c. **High Bandwidth Overhead:** Many modern web frameworks leverage heavy client-side scripts and unoptimized media delivery channels that perform poorly under standard 3G/4G regional mobile networks common in rural municipalities.

3.3 Notable Research Contributions

Research in crop recommendation has widely demonstrated the effectiveness of ensemble learning methods, particularly Random Forest classifiers, in predicting suitable crops from soil and environmental parameters such as N, P, K, temperature, humidity, pH, and rainfall, with studies consistently reporting classification accuracies exceeding 90%. For fertilizer prediction, Decision Tree-based classifiers have been frequently employed due to their interpretability, while K-Nearest Neighbors and Support Vector Machines have also shown competitive performance across multiple crop types and soil categories. Yield prediction has been approached as a regression problem in the majority of existing literature, with Random Forest Regressors and Linear Regression models applied to input features including cultivated area, rainfall, temperature, pesticide usage, and crop type using datasets sourced from FAO and World Bank agricultural records.

A limited but growing body of work has explored the integration of multiple agricultural prediction models into unified platforms, demonstrating that combining crop, fertilizer, and yield prediction within a single interface significantly improves practical utility by allowing users to move seamlessly from crop selection to input planning to yield estimation within one workflow. The findings from these contributions collectively validate the technical approach, input feature selections, and system architecture decisions adopted by Kishan Sahayatri and described in the subsequent chapters of this documentation.

CHAPTER 4: METHODOLOGY

4.1 Requirement Analysis

Requirement analysis identifies what the system must provide and how it should behave. Requirements are divided into functional and non-functional categories.

4.1.1 Functional Requirements

ID	Functional Requirement
FR1	User can register an account with name, username, email, and password.
FR2	User can log into the system using username and password.
FR3	User can enter crop prediction parameters (N, P, K, temperature, humidity, pH, rainfall).
FR4	System validates crop prediction inputs before forwarding.
FR5	System sends crop prediction data to the FastAPI ML service.
FR6	System displays the predicted crop class to the user.
FR7	User can enter yield prediction parameters (Area, Item, rainfall, pesticide, temperature).
FR8	System sends yield input to the FastAPI ML service.
FR9	System displays the estimated crop yield in kg/ha.
FR10	Backend manages application data using REST APIs.
FR11	MongoDB stores user accounts and prediction history.

FR12	FastAPI loads and executes trained ML models for inference.

Table 3: Functional Requirements

4.1.2 Non-Functional Requirements

Requirement	Description
Performance	The application should return predictions within a reasonable response time.
Usability	The interface should be simple enough for users without technical knowledge.
Reliability	The system should provide consistent responses for valid inputs.
Security	User authentication should use JWT; passwords should be hashed using encrypt.
Maintainability	Frontend, backend, database, and ML services should remain modular.
Scalability	The architecture should allow additional ML models and features to be added.
Availability	The web application and ML API should be available when the system is deployed.

Table 4: Non-Functional Requirements

4.2 System Design

4.2.1 System Architecture

The proposed architecture consists of four major layers:

Layer 1 :Presentation Layer (React): Provides the user interface, forms, input validation, dashboard, prediction result display, and communication with the backend APIs.

Layer 2 :Application Layer (Node.js + Express.js): Handles user authentication using JWT, defines application API routes, processes requests, communicates with MongoDB for data storage, and forwards prediction requests to FastAPI.

Layer 3 :Database Layer (MongoDB): Stores user information, prediction history, and other application-related data in a document-based NoSQL database.

Layer 4 :Machine Learning Layer (Python + FastAPI + Scikit-learn + Joblib): Loads trained ML models at startup, receives prediction inputs via HTTP, performs preprocessing, generates predictions, and returns results as JSON responses.

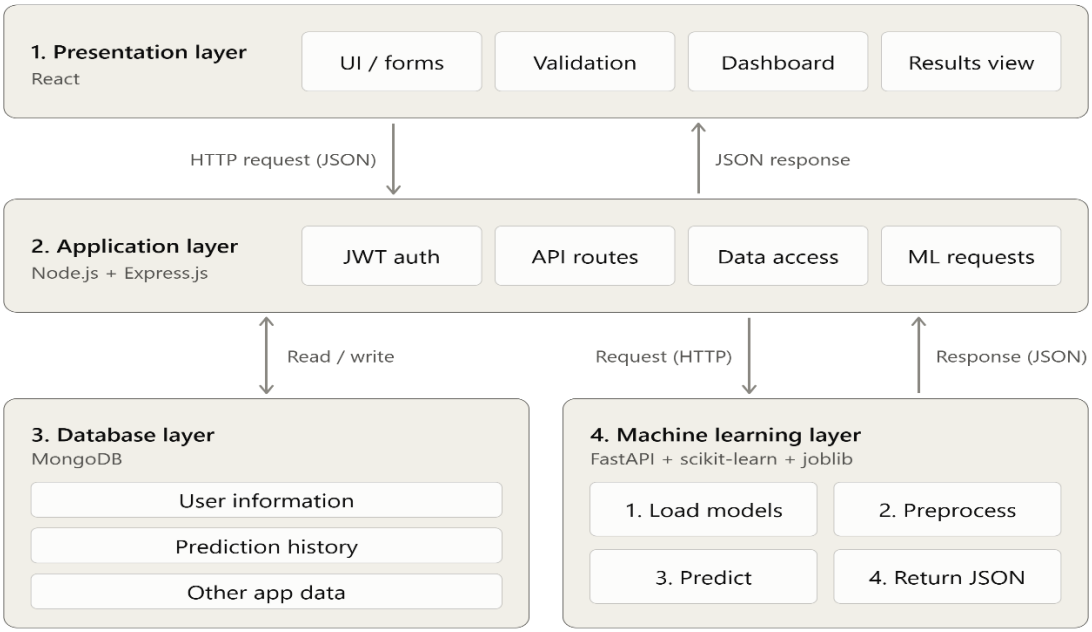


Figure 1: System Architecture

4.2.2 Machine Learning Pipeline

Crop Prediction Pipeline:.

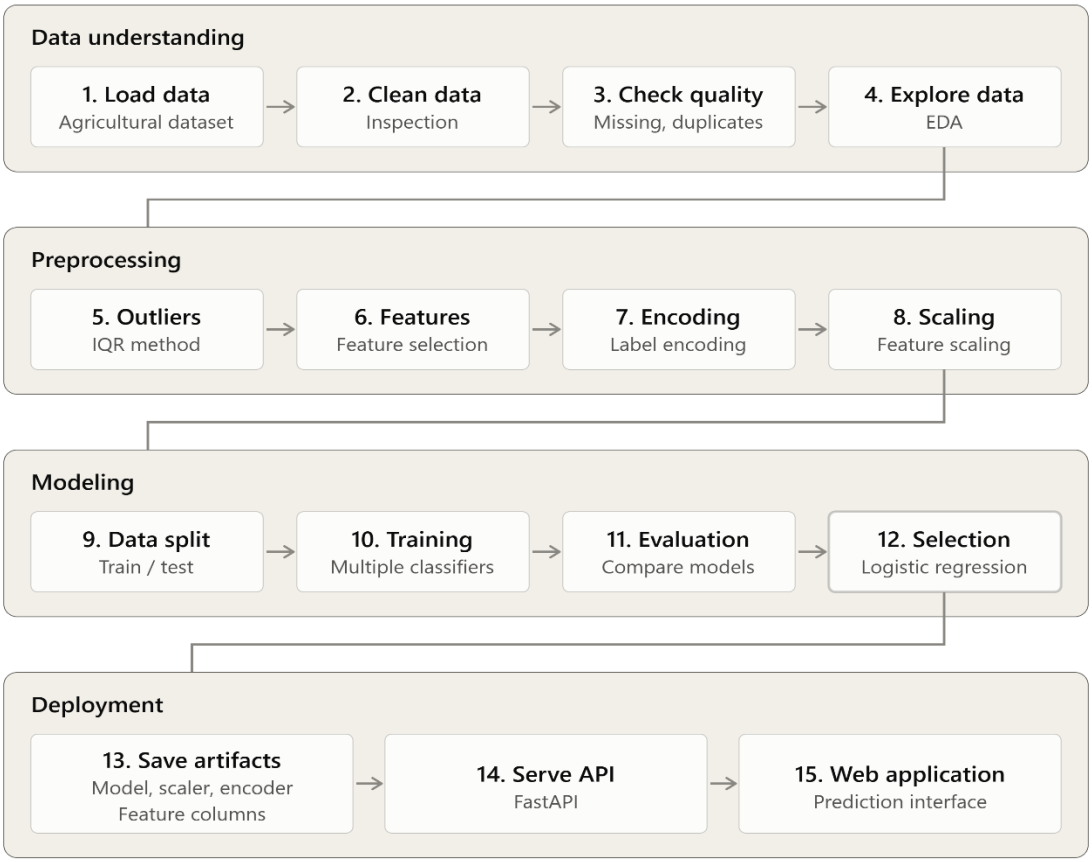


Figure 2: Crop Prediction Pipeline

Yield Prediction Pipeline:

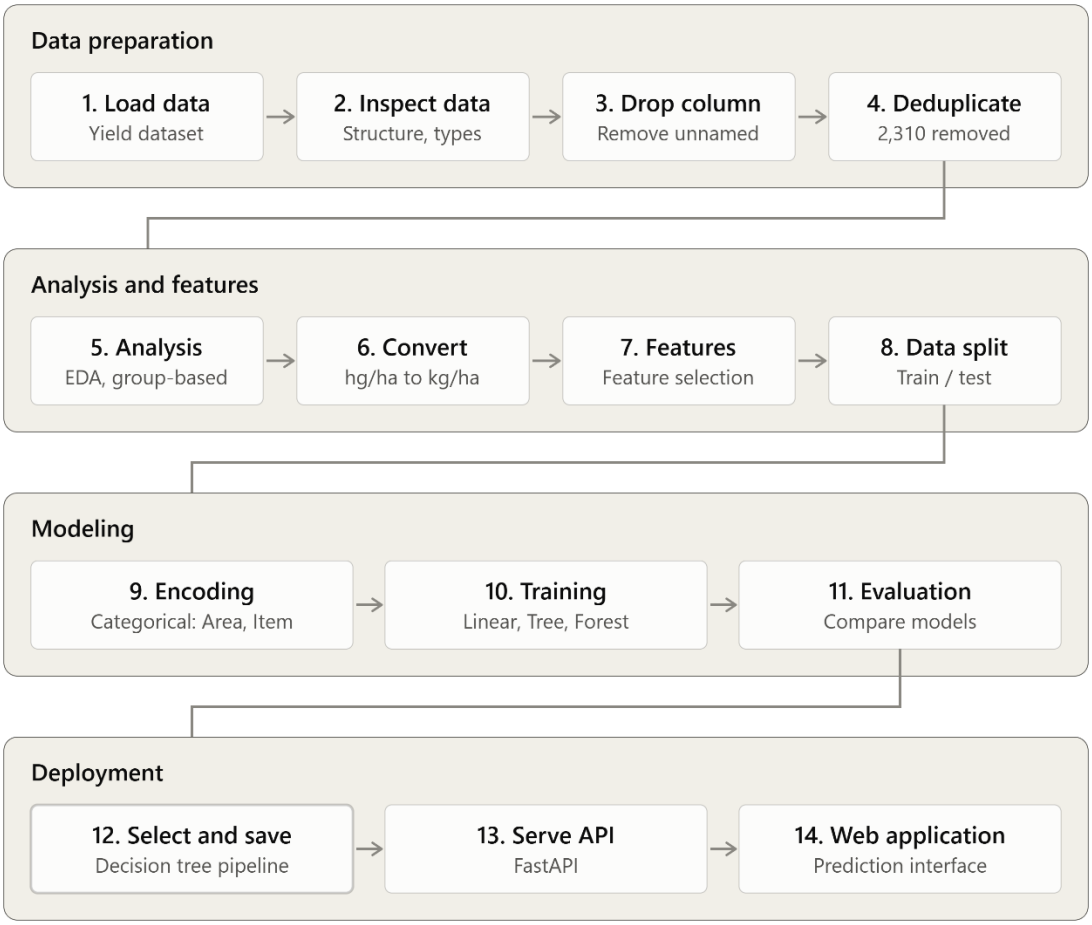


Figure 3: Yield Prediction Pipeline

4.2.3 Use-Case Diagram

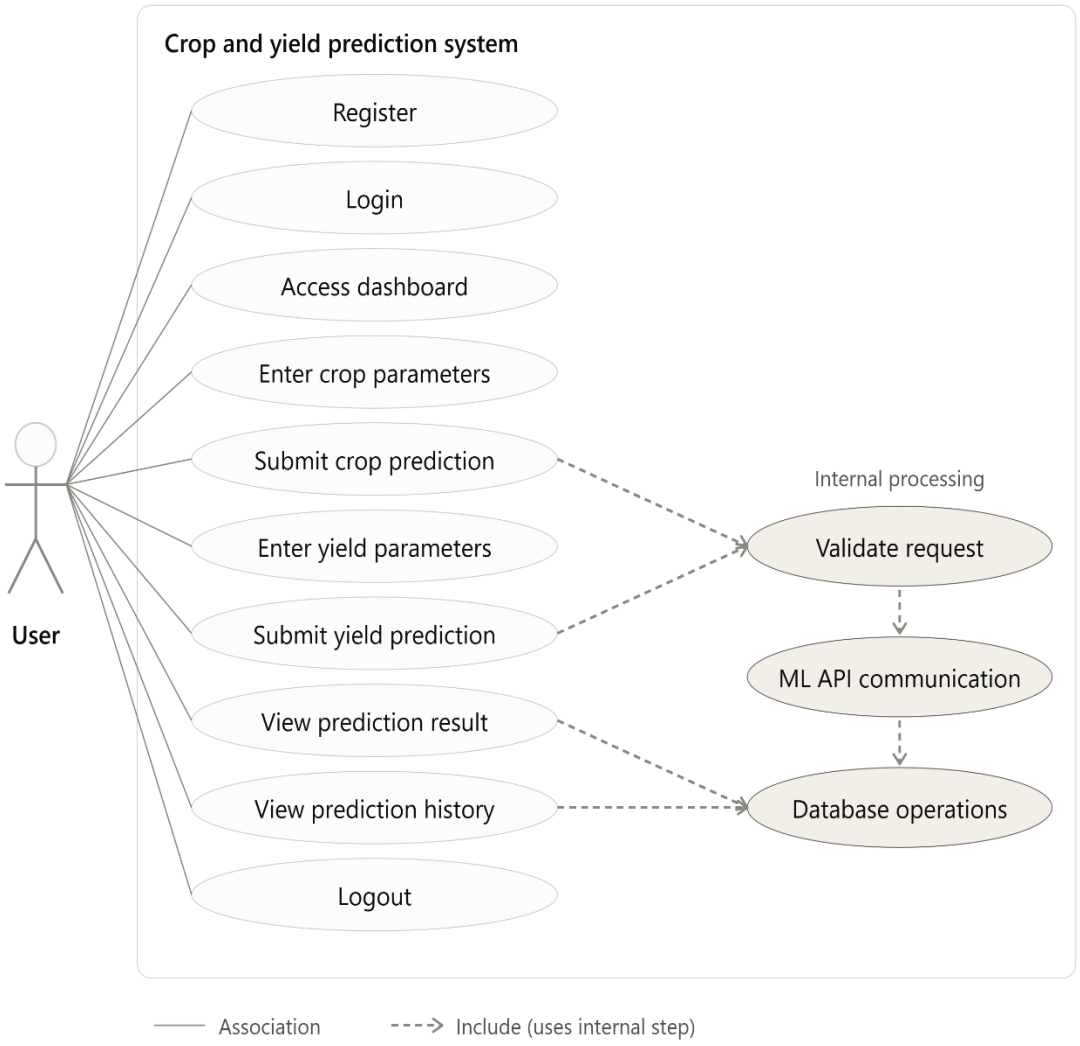


Figure 4: Use-Case Diagram

4.2.4 Sequence Diagram

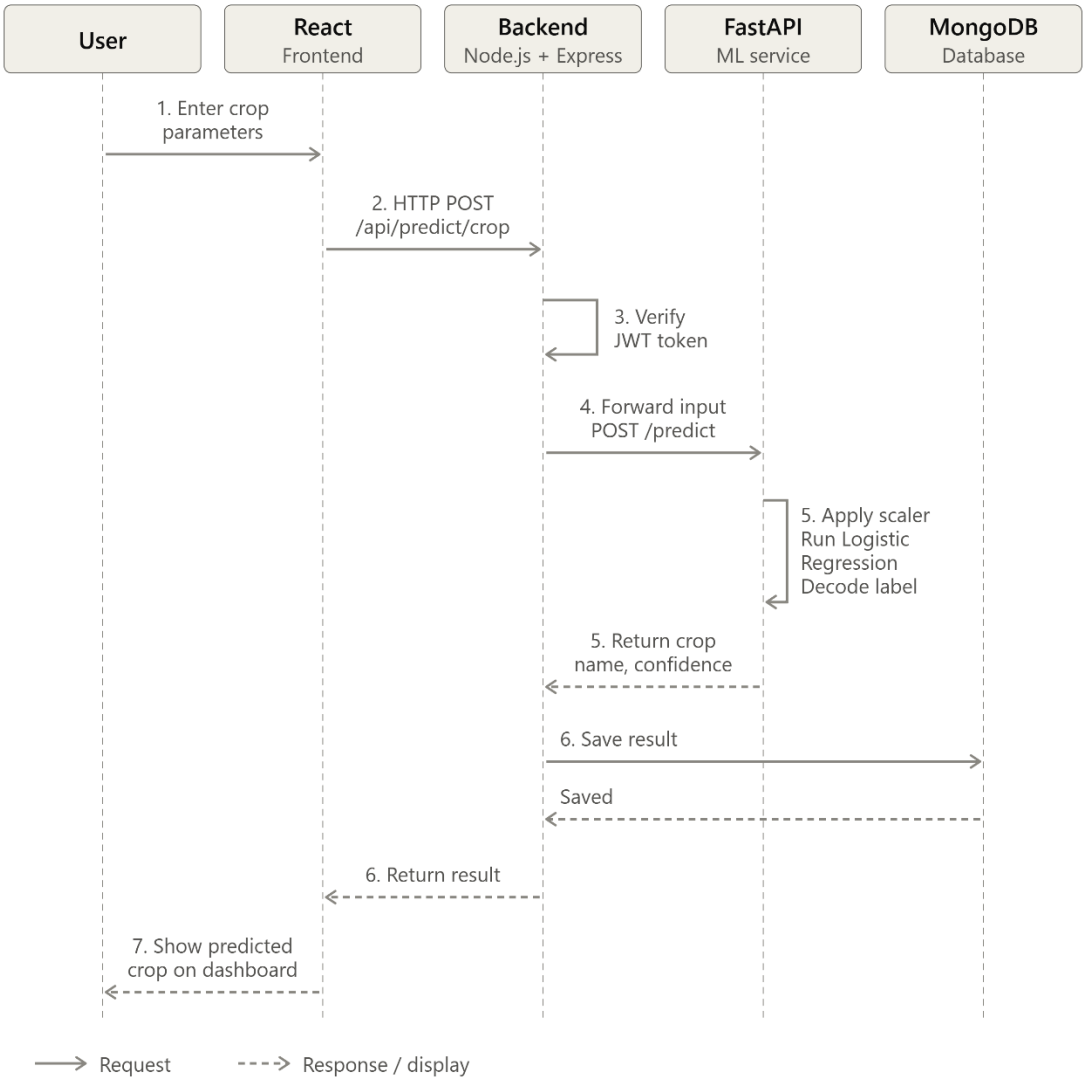


Figure 5: Sequence Diagram

4.2.5 Data Flow Diagram

Context-Level DFD:

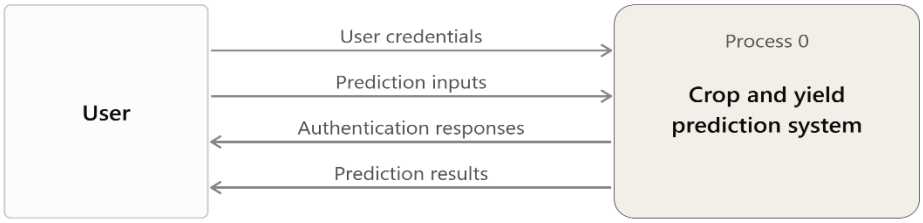


Figure 6: Context-Level DFD

Level-1 DFD:

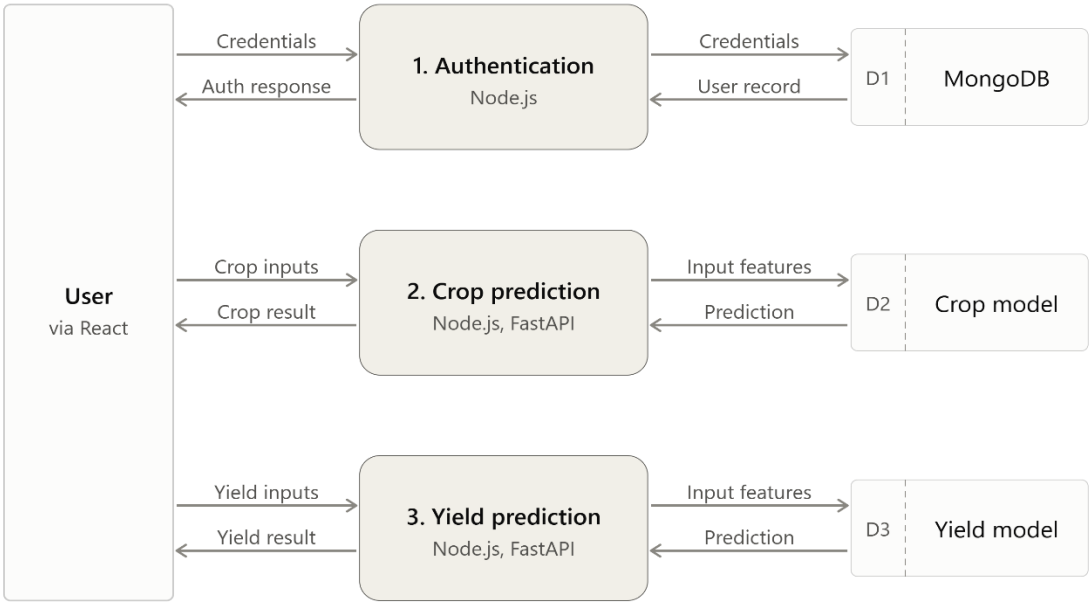


Figure 7: Level-1 DFD

4.2.6 Algorithms

Algorithm 1: Crop Prediction

Input: N, P, K, temperature, humidity, pH, rainfall

Output: Predicted crop name

1. Start.
2. Receive crop prediction inputs from the frontend through the Node.js backend.
3. Validate the input values using express-validator middleware.
4. Forward validated inputs to FastAPI via HTTP POST.
5. Arrange inputs according to the trained feature order: [N, P, K, temperature, humidity, pH, rainfall].
6. Load the saved scaler (crop_scaler.pkl).
7. Transform the input features using the scaler.
8. Load the Logistic Regression model (crop_model.pkl).
9. Generate the numerical class prediction.
10. Load the Label Encoder (crop_label_encoder.pkl).
11. Convert the numerical class into the crop name.
12. Return the predicted crop name and confidence score as JSON.
13. Node.js backend saves the prediction to MongoDB.
14. React frontend displays the result. End.

Algorithm 2: Yield Prediction

Input: Area, Item, average_rain_fall_mm_per_year, pesticides_tonnes, avg_temp

Output: Predicted yield in kg/ha

15. Start.
16. Receive yield prediction inputs from the frontend through the Node.js backend.
17. Validate the inputs using express-validator middleware.
18. Verify that Area and Item belong to the trained categories.
19. Construct the input DataFrame with the correct column names.

20. Load the saved Decision Tree pipeline (decision_tree_model.pkl).
21. The pipeline applies OneHotEncoding to Area and Item internally.
22. Generate the yield prediction.
23. Return the predicted yield in kg/ha as a JSON response.
24. Node.js backend saves the prediction to MongoDB.
25. React frontend displays the estimated yield. End.

4.2.7 System Flowchart

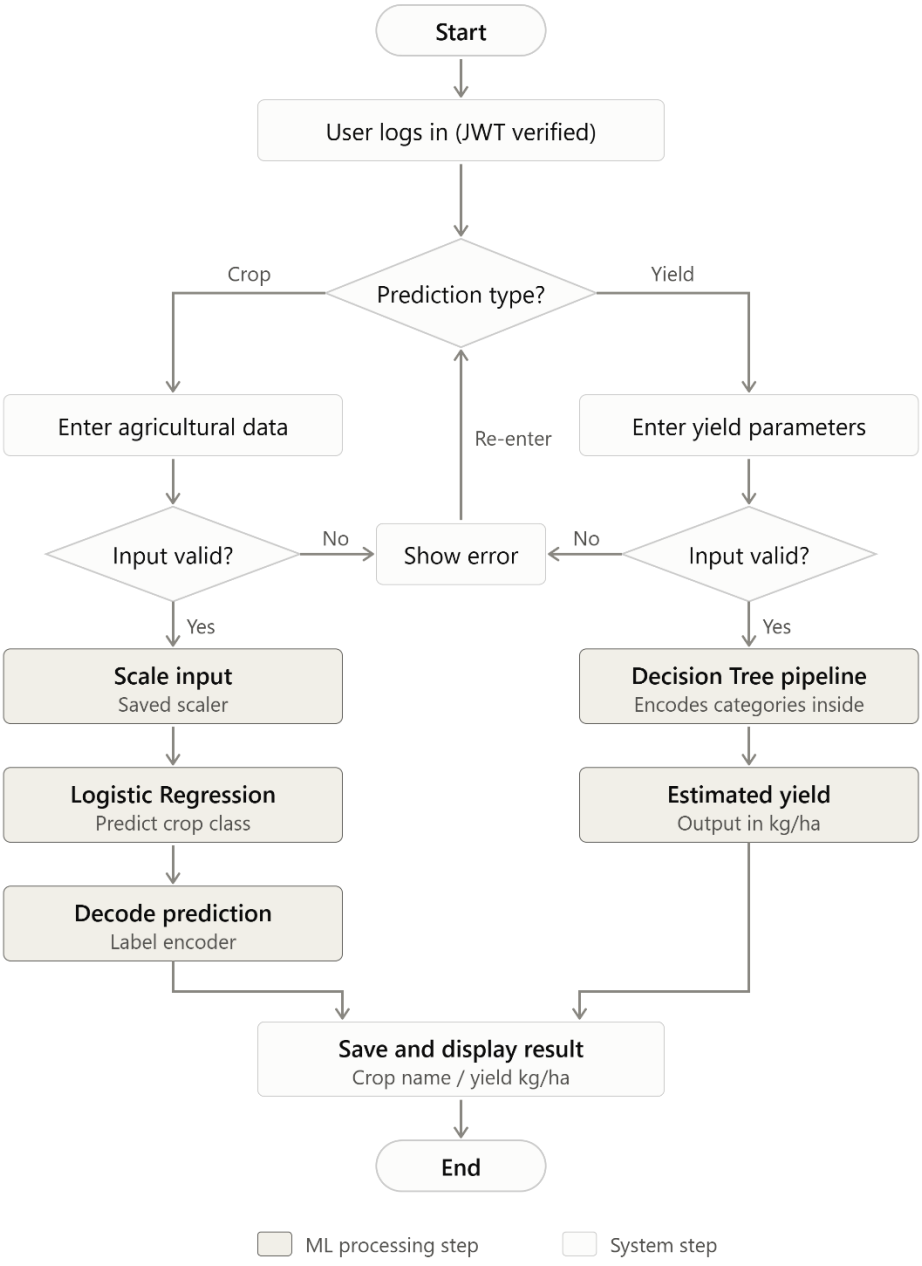


Figure 8: System Flowchart

4.3 Tools and Technologies

Category	Technology	Purpose
Frontend	React	User interface and component-based web development
Backend Runtime	Node.js	JavaScript runtime for backend server
Backend Framework	Express.js	REST API routing and middleware
Database	MongoDB + Mongoose	NoSQL document database and ODM
ML API Framework	FastAPI	Serving trained ML models as REST API
ML Language	Python	Model development and data processing
ML Library	Scikit-learn	Model training, preprocessing, evaluation
Data Processing	Pandas	Dataset loading and manipulation
Numerical Processing	NumPy	Numerical operations
Model Saving	Joblib	Serializing and loading ML models
Development Environment	Jupyter Notebook	Exploratory analysis and model training
Authentication	JWT + bcryptjs	User authentication and password hashing
API Communication	HTTP/REST	Communication between application layers
Data Format	JSON	Data exchange format

Version Control	Git/GitHub	Source code management

Table 5: Tools and Technologies Used

4.4 Development Model

An Agile/Iterative development model is used for this project because the application consists of several independently developable modules: ML development, backend development, frontend development, and integration. Each iteration produces a testable increment.

The project is divided into the following iterations:

a. Requirement Analysis — Identify problems and system requirements.
b. Dataset Preparation — Collect and prepare crop and yield datasets.
c. ML Development — Train and evaluate crop and yield prediction models.
d. Backend Development — Develop Node.js and Express.js APIs with JWT authentication.
e. Frontend Development — Develop React pages and prediction forms.
f. ML API Integration — Integrate trained models with FastAPI.
g. System Integration — Connect React, Node.js, MongoDB, and FastAPI.
h. Testing — Perform functional, integration, and model testing.
i. Documentation and Deployment.

4.5 Model Training and Evaluation Strategy

4.5.1 Crop Prediction Dataset

The crop prediction notebook loads the file crop_fertilizer_recommendation_dataset.csv. The dataset is described below:

Property	Value
Dataset file	crop_fertilizer_recommendation_dataset.csv
Original rows	2,200
Original columns	13
Records per crop class	100
Number of crop classes	22
Duplicate records	0 (none found)
Missing values	None found
Final features used	N kg/ha, P kg/ha, K kg/ha, temperature, humidity, pH, rainfall
Target variable	label (crop class)

Table 6: Crop Prediction Dataset Description

4.5.2 Crop Data Preprocessing

The notebook performs the following preprocessing steps:

a. Load and inspect the dataset.
b. Remove columns not required for prediction: recommendation_note, fertilizer, N_status, P_status, and K_status.
c. Rename columns for consistency.
d. Check for missing values (none found).
e. Check for duplicate records (none found).
f. Perform exploratory data analysis.

g. Perform outlier analysis using the IQR method.
h. Perform correlation analysis.
i. Apply Label Encoding to the target variable.
j. Apply Feature Scaling to the input features.
k. Perform train-test split.

4.5.3 Crop Prediction Findings

Area	Key Finding
Dataset	2,200 records and 13 columns originally; 8 columns kept (7 numeric features and the crop label).
Data quality	No missing values and no duplicate records were found.
Class balance	22 crop classes with exactly 100 records each, so the dataset is perfectly balanced.
Columns removed	recommendation_note (identical in all 2,200 rows), fertilizer, and the N, P, K status columns, as they are not needed for crop prediction.
pH outliers	57 outliers by the IQR method (limits 4.54 to 8.35), mostly in mothbeans and chickpea. They were retained.
Phosphorus outliers	138 outliers by the IQR method (limits -32 to 128), mainly in grapes and apple. They were retained.
Correlation	Phosphorus and potassium are strongly correlated (0.74). All other feature pairs are weakly correlated (up to 0.23 in magnitude).
Preprocessing	Crop labels encoded into 22 classes, features standardized with StandardScaler, and an 80/20 train-test split (1,760 train, 440 test).
Model accuracy	Naive Bayes 99.55%, Random Forest 99.32%, Decision Tree 98.64%, SVM 96.82%, Logistic Regression 96.36%, KNN 95.23%.
Overfitting check	Naive Bayes: train 99.49%, test 99.55%. Random Forest: train 100.00%, test 99.32%.
Weakest classes	Logistic Regression scored lowest F1 on rice (0.83) and jute (0.85). All other crops scored 0.89 to 1.00.

Deployed model	Logistic Regression, saved with its scaler, label encoder, and feature columns as .pkl files.

Table 7: Crop prediction Findings

4.5.4 Crop Model Evaluation

Model	Test Accuracy	F1 Score
Logistic Regression	96.36%	96.35%
K-Nearest Neighbors	95.23%	95.18%
Naive Bayes	99.55%	99.54%
Decision Tree	98.64%	98.63%
Support Vector Machine	96.82%	96.80%
Random Forest	99.32%	99.32%

Table 8: Crop Prediction Model Results

Among the classification algorithms evaluated, Naive Bayes (99.55%) and Random Forest (99.32%) achieved the highest test accuracies, while Logistic Regression achieved 96.36%. However, accuracy values this close to 100% on a small, clean, and evenly balanced dataset (2,200 records, 22 classes, 100 records per class) should be interpreted with caution. Such results may indicate that the models are fitting the specific characteristics of this dataset rather than learning patterns that will hold for real-world inputs. Random Forest, for example, reached 100% training accuracy, which suggests a tendency to memorize the training data.

For this reason, model selection was not based on accuracy alone. Logistic Regression was chosen for deployment because it is better suited to a practical prediction service, for the following reasons:

a. It is a simple and stable model that is less prone to overfitting than complex ensemble methods.

b. Its accuracy of 96.36% is still high, so a marginally lower score is an acceptable trade-off for better generalization.

c. It produces class probabilities, which allows the system to return a confidence score along with the predicted crop.

d. It is fast to load and run, which suits real-time prediction through FastAPI.

e. It is easy to interpret, retrain, and maintain.

Based on these considerations, Logistic Regression was selected as the crop prediction model and deployed through FastAPI. The trained model and its supporting objects were saved for use during prediction:

a. **crop_model.pkl:** the trained Logistic Regression model.

b. **crop_scaler.pkl**: the scaler used to standardize input features.

c. **crop_label_encoder.pkl**: the encoder used to convert predicted classes back into crop names.

d. **crop_feature_columns.pkl:** the list of feature columns, which keeps the input order consistent.

4.5.5 Yield Prediction Dataset

Property	Value
Dataset file	yield_df.csv
Original rows	28,242
Original columns	8
Duplicate records removed	2,310
Final records	25,932
Input features	Area, Item, average_rain_fall_mm_per_year, pesticides_tonnes, avg_temp
Target variable	kg/ha_yield (converted from hg/ha)
Year column	Excluded from final model features

Table 9: Yield Prediction Dataset Description

4.5.6 Yield Data Preprocessing

a. The preprocessing steps performed in the yield notebook are:
b. Load and inspect the dataset.
c. Remove the Unnamed: 0 column.
d. Check missing values.
e. Identify and remove 2,310 duplicate records.
f. Perform descriptive and group-based analysis.
g. Convert yield from hg/ha to kg/ha (multiply by 0.1).
h. Remove the original hg/ha_yield column.

i. Separate input and target variables.
j. Perform train-test split.
k. Apply categorical preprocessing (OneHotEncoding) to Area and Item using a pipeline. Train regression models.

4.5.7 Yield Prediction Findings

Area	Key Finding
Dataset	28,242 records and 8 columns originally; the unnamed index column was removed.
Data quality	No missing values. 2,310 duplicate records were removed, leaving 25,932 records.
Coverage	Records from 101 areas, 10 crop items, and the years 1990 to 2013.
Crop records	Potatoes (4,276), Maize (4,121), and Wheat (3,857) had the most records, and Plantains and others (556) the fewest (counted before duplicate removal).
Yield by crop	Potatoes had the highest average yield (about 19,867 kg/ha), followed by Cassava (14,565 kg/ha). Soybeans (1,699 kg/ha) and Sorghum (1,910 kg/ha) had the lowest.
Yield trend	Average yield rose steadily from about 6,672 kg/ha in 1990 to 9,036 kg/ha in 2013 (about 35%), while average temperature stayed stable at about 20 to 21°C.

Area	Key Finding
Climate ranges	Rainfall ranged from 51 to 3,240 mm/year (mean 1,147). Average temperature ranged from 1.30 to 30.65°C (mean 20.42).
Pesticide use	Highly skewed: mean 34,783 tonnes against a median of 15,373 and a maximum of 367,778. Yams had the highest average use and Plantains and others the lowest.
Preprocessing	Yield converted from hg/ha to kg/ha (× 0.1). Year was excluded. Inputs are Area, Item, rainfall, pesticides, and temperature.
Encoding and split	Area and Item are one-hot encoded inside the pipeline, and the numeric features pass through unchanged. The split is 80/20 (20,745 train, 5,187 test).
Linear Regression	MAE 3193.10, RMSE 4881.81, R^2 0.6712. Weak fit.
Decision Tree	MAE 732.15, RMSE 1830.88, R^2 0.9538 (max depth 20).
Random Forest	MAE 630.09, RMSE 1556.54, R^2 0.9666 (200 trees). Best overall.
Deployed model	Decision Tree pipeline saved as decision_tree_model.pkl, which includes the preprocessing.
Sample prediction	Australia, Rice, 1500 mm rainfall, 400 tonnes pesticides, 14°C gave 6,061 kg/ha.

Table 10: Yield Prediction Findings

4.5.8 Yield Model Evaluation

Model	MAE	RMSE	R^2
Linear Regression	3193.10	4881.81	0.6712
Decision Tree Regression	732.15	1830.88	0.9538
Random Forest Regression	630.09	1556.54	0.9666

Table 11: Yield Prediction Model Results

Among the three regression models tested, Linear Regression performed poorly (MAE 3193.10, RMSE 4881.81, R^2 0.6712), so it could not capture the relationship between the inputs and crop yield. Random Forest gave the best scores (MAE 630.09, RMSE 1556.54, R^2 0.9666), but the Decision Tree came very close (MAE 732.15, RMSE 1830.88, R^2 0.9538). It explains about 95% of the variation in yield, and its errors are reasonable compared with the typical yield values in the dataset.

The Decision Tree was chosen for deployment because it gives accurate results with a much simpler and lighter model than Random Forest. It is also easier to load, faster to run, and easier to maintain. The trained model is saved as **decision_tree_model.pkl**. This is a complete pipeline that includes the categorical preprocessing for Area and Item together with the Decision Tree, so FastAPI can generate a prediction with a single pipeline.predict() call, without repeating the preprocessing steps.

4.6 Testing Strategy

Testing is performed at multiple levels to ensure the system functions correctly as both individual components and as an integrated system.

4.6.1 Unit Testing

Individual functions and components are tested independently, including login validation, form validation on the frontend, API request handling in the backend, input preprocessing in FastAPI, and prediction functions.

4.6.2 Integration Testing

Integration testing verifies correct communication between: React and Node.js, Node.js and MongoDB, Node.js and FastAPI, and FastAPI and ML models.

4.6.3 System Testing

The complete application is tested as a single system, verifying end-to-end workflows from user input through prediction result display.

4.6.4 Test Cases

Test ID	Test Case	Expected Result
TC01	Register with valid name, username, email, password	Account created, 201 response
TC02	Login with valid credentials	JWT token returned, user redirected to dashboard
TC03	Login with invalid credentials	Error message displayed
TC04	Submit valid crop inputs (7 parameters)	Crop prediction returned and displayed
TC05	Submit incomplete crop inputs	Validation error displayed
TC06	Submit valid yield inputs (5 parameters)	Yield prediction in kg/ha returned

TC07	Submit invalid yield input (unknown Area)	Validation error from FastAPI returned
TC08	FastAPI service unavailable	Appropriate API error returned to frontend
TC09	Access protected route without token	401 Unauthorized response returned
TC10	Prediction response received successfully	Result displayed correctly in React

Table 12: Testing Cases

4.7 Deployment and Maintenance

The application consists of three main deployable components: (1) React Frontend, (2) Node.js + Express.js Backend, and (3) Python + FastAPI ML Service, along with the MongoDB database.

During development, the React frontend runs on port 5173 (Vite dev server), the Node.js backend runs on port 5000, the crop prediction FastAPI service runs on port 8001, and the yield prediction FastAPI service runs on port 8002.

Maintenance activities include: updating datasets; retraining models when new data becomes available; monitoring prediction performance; updating dependencies; fixing application errors; improving the user interface; adding new crops or agricultural modules; improving API security; and maintaining the MongoDB database.

4.8 Project Management

The project activities are coordinated across three team members, each responsible for a different system layer: Prasan Poudel (AI/ML), Anish Thapa (Frontend), and Aashutosh Lama (Backend).

4.8.1 Gantt Chart

Activity	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10
Problem Identification	✓									
Requirement Analysis	✓	✓								
Literature Review	✓	✓								
Dataset Collection		✓	✓							
Data Preprocessing			✓	✓						
ML Model Development			✓	✓						
Backend Development				✓	✓					
Frontend Development				✓	✓	✓				
FastAPI Integration					✓	✓				
System Integration						✓	✓			
Testing							✓	✓		
Documentation								✓	✓	
Final Deployment									✓	✓

Table 13: Project Development Schedule (Gantt Chart)

CHAPTER 5: SNAPSHOTS

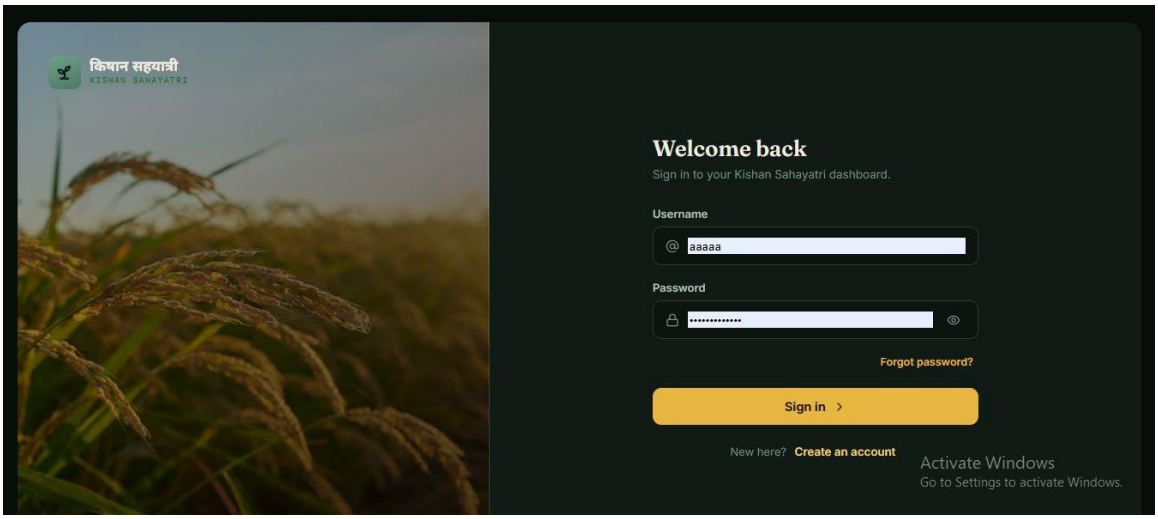


Figure 9:Login Page

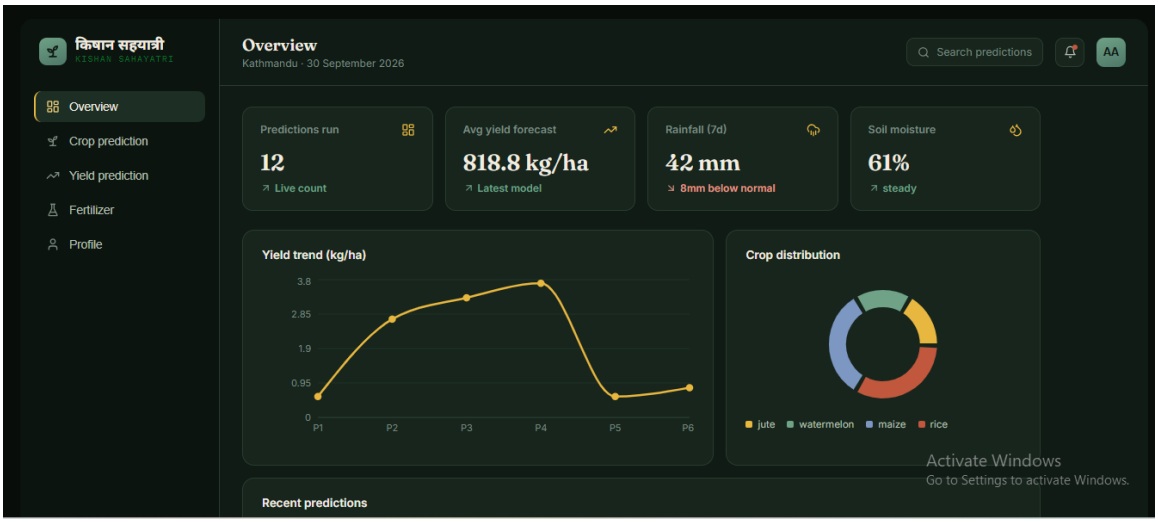


Figure 10: Main Page

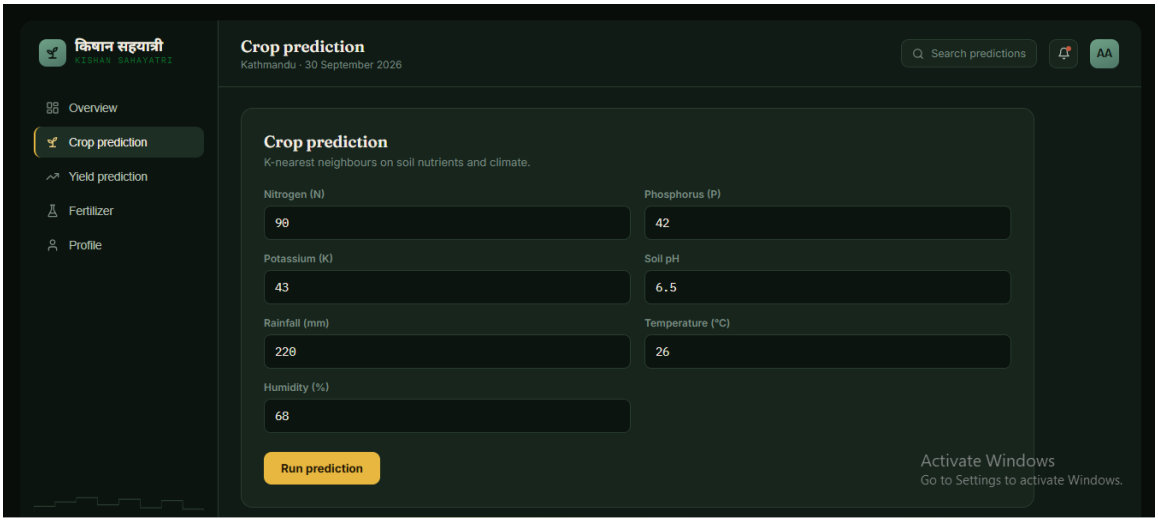


Figure 11: Crop Prediction

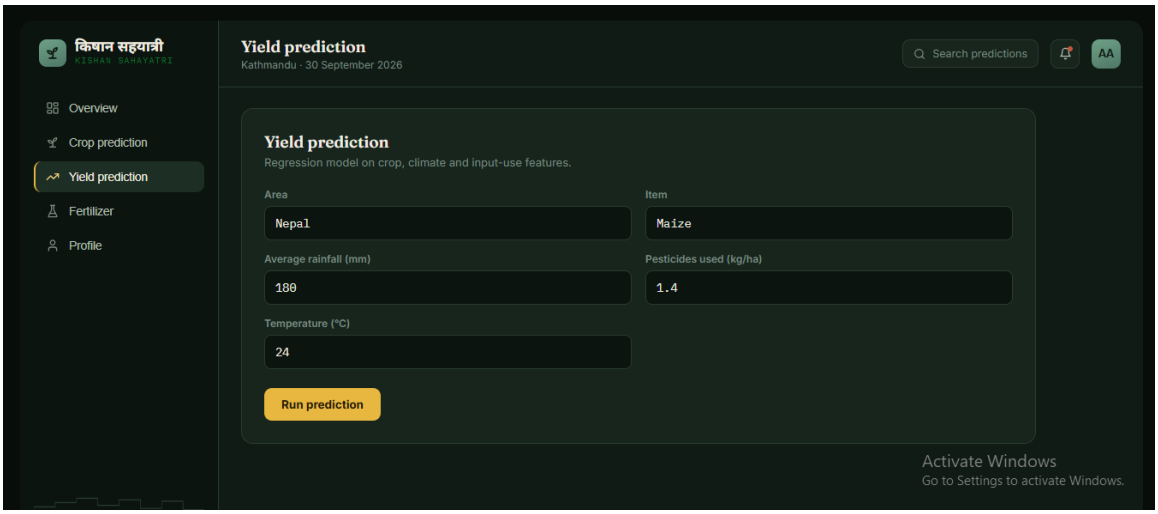


Figure 12: Yield Prediction

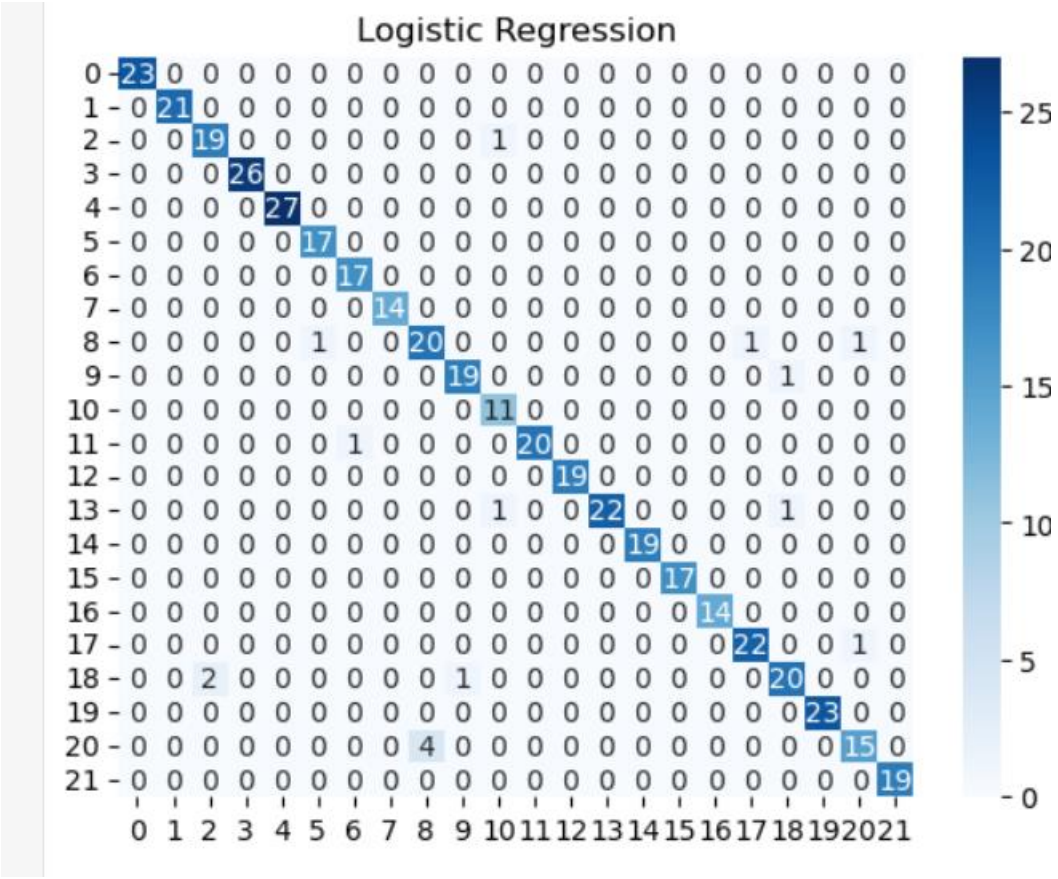


Figure 13: Confusion Matrix(Crop Prediction)

CHAPTER 6: CONCLUSION

Kishan Sahayatri is a machine-learning-based agricultural web application designed to make agricultural predictions accessible through a web interface. The project combines machine learning with modern web technologies to create an integrated, layered system.

The crop prediction component treats crop selection as a classification problem. The dataset contains 2,200 records covering 22 crop classes and uses nitrogen, phosphorus, potassium, temperature, humidity, pH, and rainfall as input features. Several classification algorithms were evaluated, including Logistic Regression, KNN, Naive Bayes, Decision Tree, SVM, and Random Forest. The results demonstrate that the dataset supports high-performing crop classification. Logistic Regression (96.36% test accuracy) is selected as the deployed model due to its performance and interpretability, and is exported with its corresponding scaler, label encoder, and feature-column objects for FastAPI integration.

The crop yield prediction component treats yield estimation as a regression problem. After removing 2,310 duplicate records, the yield dataset contains 25,932 records. Linear Regression, Decision Tree Regression, and Random Forest Regression were evaluated using MAE, RMSE, and R^2. The Decision Tree achieved an R^2 of 0.9538 and is saved as a pipeline for deployment, combining categorical preprocessing and the regression model in a single serialized object.

The web application architecture separates the system into four layers: the React frontend (presentation), the Node.js/Express.js backend (application), MongoDB (database), and FastAPI (machine learning service). This layered architecture ensures that each component can be developed, tested, and maintained independently.

The backend provides JWT-based user authentication, RESTful API endpoints for crop and yield predictions, prediction history management in MongoDB, and proper input validation and error handling. The backend acts as the integration point between the React frontend and the FastAPI ML services.

The proposed architecture also provides a foundation for future expansion. Additional agricultural services such as fertilizer recommendation, disease prediction, weather integration, soil information, and agricultural advisory features can be incorporated into the existing system architecture.

Overall, the project demonstrates how machine learning can be integrated with a modern MERN web application to provide accessible agricultural prediction services. The system serves as a decision-support tool while recognizing that real-world agricultural outcomes depend on many factors beyond those represented in the current datasets.

References

[1] Gautam Patil. (n.d.). *Crop and fertilizer recommendation using machine learning* [Kaggle notebook]. Kaggle. Kaggle

[2] Figolm10. (n.d.). *Crop-yield-prediction* [Kaggle notebook]. Kaggle. Kaggle

[3] van Klompenburg, T., Kassahun, A., & Catal, C. (2020). Crop yield prediction using machine learning: A systematic literature review. *Computers and Electronics in Agriculture, 177*, 105709. https://doi.org/10.1016/j.compag.2020.105709

[4] Iniyan, S., Varma, A. V., & Naidu, C. T. (2023). Crop yield prediction using machine learning techniques. *Advances in Engineering Software, 175*, 103326.

[5] National Agricultural Research Council. (2024). *Soil data*. National Soil Science Research Center. NARC Soil Database